

LeadOS RM Portal Screen Requirements Document for Shivansh

1. Purpose

The RM Portal is the working dashboard for Relationship Managers and sales users.

For the MVP, Shivansh should build the complete RM Portal UI using static mock data first. Dev will connect real APIs later. The UI should be structured exactly as if real backend APIs are already integrated.

The RM Portal must prove one of the most important requirements of the theme:

Hot leads must be handed off to a human RM with full conversation context, not just a name and phone number.

The RM should be able to quickly understand:

Who is this lead?
Why is this lead Hot?
What did the AI discuss?
Which objection came up?
Was the objection resolved?
What should I do next?

2. Tech Stack

Use the same standard stack as the Admin Portal.

- Vite React
- TypeScript
- TanStack Query
- TanStack Router
- Zustand
- React Hook Form plus Zod
- Tailwind CSS
- shadcn/ui
- lucide-react for icons
- Recharts or any clean React chart library if needed

3. Development Rule

For now, build everything using static data.

Important rules:

- Do not hardcode data directly inside UI components
- Create mock data files
- Create service functions that return mock API responses
- Use TanStack Query even for mock API calls
- Keep response shapes close to the real API contracts
- Build loading states, empty states, and error states
- Dev will later replace mock services with real API calls

Suggested structure:

```
src/  
  routes/  
  components/  
  features/  
    rm/  
      dashboard/  
      leads/  
      tasks/  
      followups/  
      notifications/  
      feedback/  
  services/  
    rmApi.ts  
  mocks/  
    rm.mock.ts  
  store/  
    authStore.ts  
    uiStore.ts  
  types/  
    rm.types.ts
```

4. RM Portal Layout

The RM Portal should be simpler than the Admin Portal. It should focus on action, not management.

Main layout components

Required layout:

- Sidebar navigation
- Top header
- Page title area
- Main content area
- Notification icon
- RM profile menu

- Client name display
- Logout action

Sidebar items

Use these sidebar items:

Dashboard
Hot Leads
My Leads
Follow Ups
Notifications
Feedback

Optional if time allows:

Performance

Header

Header should show:

Current page title
Client name: FinPartner Pro
Notification icon
RM avatar or initials
User menu
Logout option

5. Routes

Use TanStack Router.

Required routes:

/rm/login
/rm
/rm/dashboard
/rm/hot-leads
/rm/leads
/rm/leads/\$leadId
/rm/followups
/rm/notifications
/rm/feedback

Optional route:

/rm/performance

Default redirect:

/rm → /rm/dashboard

6. Shared Types

Use these common types in the UI.

```
type UserRole = "admin" | "rm";

type LeadStatus =
  | "new"
  | "pending_contact"
  | "conversation_completed"
  | "assigned_to_rm"
  | "follow_up_scheduled"
  | "converted"
  | "closed";

type LeadClassification = "hot" | "warm" | "cold";

type ConversationChannel = "chat" | "voice";

type RMTaskStatus = "pending" | "in_progress" | "converted" | "closed";

type FollowUpStatus = "ready" | "opened" | "sent" | "cancelled";

type ObjectionResolutionStatus =
  | "resolved"
  | "partially_resolved"
  | "unresolved";
```

7. Page 1: RM Login

Purpose

Allow RM to login.

During static phase, login should set mock auth state in Zustand and redirect to RM dashboard.

Route

/rm/login

Components

- Login card
- Email input
- Password input
- Submit button
- Demo credentials hint

- Loading state
- Error message

Validation

Use React Hook Form plus Zod.

Rules:

```
Email is required
Email must be valid
Password is required
```

Mock service

```
loginRM(payload)
```

Request:

```
{
  "email": "rm@leados.demo",
  "password": "password123"
}
```

Response:

```
{
  "success": true,
  "data": {
    "accessToken": "mock_access_token",
    "refreshToken": "mock_refresh_token",
    "user": {
      "id": "user_2",
      "clientId": "client_1",
      "name": "Shivansh Mishra",
      "email": "rm@leados.demo",
      "role": "rm"
    },
    "client": {
      "id": "client_1",
      "name": "FinPartner Pro",
      "slug": "finpartnerpro"
    }
  }
}
```

Later real API:

```
POST /auth/login
```

8. Page 2: RM Dashboard

Purpose

The RM Dashboard should show the RM what needs attention right now.

It should not feel like a management dashboard. It should feel like a work queue.

Route

/rm/dashboard

Sections

- KPI cards
- Hot lead queue preview
- Follow ups due
- Recent handoffs
- Notifications preview
- Personal performance summary optional

KPI cards

Show:

Assigned Hot Leads
Pending Tasks
Follow Ups Due
Converted Leads
Closed Leads
Average Lead Score optional

Each card should include:

- Value
- Label
- Icon
- Small helper text

Example:

Assigned Hot Leads: 8
Follow Ups Due: 3
Converted Leads: 5

Hot lead queue preview

Show top 5 assigned Hot leads.

Each card should show:

Lead name
Score
Language
Main objection
Recommended next action
Open button

Follow ups due section

Show Warm follow ups or scheduled follow ups.

Each row/card should show:

Lead name
Channel
Status
Scheduled time
Open follow up button

Recent handoffs

Show newly assigned Hot leads from AI.

Fields:

Lead name
Score
Why Hot
Assigned time
Open detail button

Mock service

`getRMDashboardOverview()`

Later real APIs:

`GET /rm/tasks`
`GET /notifications`
`GET /followups`
`GET /leads`

Mock response:

```
{
  "success": true,
  "data": {
    "overview": {
      "assignedHotLeads": 8,
```

```

    "pendingTasks": 6,
    "followUpsDue": 3,
    "convertedLeads": 5,
    "closedLeads": 2,
    "averageLeadScore": 76
  },
  "hotLeads": [
    {
      "id": "lead_1",
      "name": "Amit Sharma",
      "phone": "9876543210",
      "preferredLanguage": "Hinglish",
      "classification": "hot",
      "latestScore": 82,
      "mainObjection": "Existing Broker",
      "recommendedNextAction": "Call today before 6 PM"
    }
  ],
  "followUpsDue": [
    {
      "id": "follow_1",
      "leadId": "lead_2",
      "leadName": "Priya Mehta",
      "channel": "whatsapp",
      "status": "ready",
      "message": "Hi Priya, thanks for speaking with us...",
      "waMeLink": "https://wa.me/919876543211?text=Hello"
    }
  ],
  "notifications": [
    {
      "id": "notif_1",
      "title": "New Hot Lead Assigned",
      "message": "Amit Sharma has been classified as Hot.",
      "type": "hot_lead_assigned",
      "isRead": false
    }
  ]
}

```

9. Page 3: Hot Leads

Purpose

This is the most important RM list page.

It shows all Hot leads assigned to the RM. These are the leads the RM should prioritize.

Route

/rm/hot-leads

Components

- Page header
- Search input
- Filters
- Hot leads table or card list
- Pagination
- Empty state
- Loading skeleton
- Error state

Card or table fields

Show:

Lead name
Phone
City
Role
Language
Score
Main objection
Recommended next action
Task status
Assigned time
Actions

Filters

Search by name or phone
Language
Task status
Score range optional
Date range

Sorting

Score
Assigned time
Lead name
Task status

Recommended UI

Use card layout if possible because RM needs context quickly.

Each Hot lead card should show:

A large score
Hot badge

Language badge
Main objection
Recommended next action
Open Lead button

Mock service

getRMHotLeads (params)

Later real API:

GET /rm/tasks?classification=hot

Mock response:

```
{
  "success": true,
  "data": [
    {
      "taskId": "task_1",
      "lead": {
        "id": "lead_1",
        "name": "Amit Sharma",
        "phone": "9876543210",
        "city": "Hyderabad",
        "role": "Financial Advisor",
        "preferredLanguage": "Hinglish",
        "classification": "hot",
        "latestScore": 82
      },
      "taskStatus": "pending",
      "priority": "high",
      "mainObjection": "Existing Broker",
      "recommendedAction": "Call today before 6 PM",
      "suggestedOpeningLine": "Hi Amit, I saw you asked about brokerage share and daily payouts.",
      "createdAt": "2026-05-02T10:25:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 8,
    "totalPages": 1
  }
}
```

10. Page 4: My Leads

Purpose

RM can see all assigned leads, not only Hot leads.

This includes:

Hot assigned leads
Converted leads
Closed leads
Warm leads assigned manually if any

Route

/rm/leads

Components

- Page header
- Search input
- Filter bar
- Leads table
- Pagination

Table columns

Lead name
Phone
Language
Status
Classification
Score
Recommended action
Created date
Actions

Filters

Search
Status
Classification
Language
Date range

Mock service

getRMLeads(params)

Later real API:

GET /leads

RM access rule:

Returns only leads assigned to current RM or related to current RM tasks.

11. Page 5: RM Lead Detail

Purpose

This is the most important RM screen.

It should give the RM everything needed to continue the conversation naturally.

Route

`/rm/leads/$leadId`

Main sections

- Lead profile
- AI score card
- AI summary
- Objections
- Transcript
- Recommended next action
- Suggested RM opening line
- RM task actions
- Follow up section
- RM feedback section

Lead profile section

Show:

Name
Phone
Email optional
City
Role
Network size
Preferred language
Preferred channel
WhatsApp consent
Campaign
Status
Classification

AI score card

Show:

Total score
Classification

Readiness score
Engagement score
Fit score
Score reason
Positive signals
Negative signals

Suggested UI:

- Large score number
- Hot/Warm/Cold badge
- Score breakdown cards
- Positive and negative signal chips

AI summary section

Show:

Conversation summary
Lead intent
Main concern
What benefit mattered
Next recommended action

Objections section

Show objection cards.

Each card should show:

Objection type
Lead statement
AI response summary
Resolution status

Resolution statuses:

Resolved
Partially Resolved
Unresolved

Example:

Objection: Existing Broker
Lead statement: I already have another broker.
Resolution: Resolved
AI response: Compared 100 percent brokerage and daily payouts.

Transcript section

Show the full transcript in chat format.

Each message should show:

Role
Message content
Timestamp
Language optional

For voice conversations, show transcript exactly like chat.

Recommended next action section

Show a clear action block.

Examples:

Call today before 6 PM.
Send onboarding link.
Follow up on trust concern.
Ask about network size.

Suggested RM opening line

Show a ready-to-use sentence.

Example:

Hi Amit, I saw you spoke with our AI assistant about daily payouts and brokerage share. I can quickly help you compare this with your current broker setup.

This is important because it proves handoff quality.

RM task actions

RM should be able to:

Mark In Progress
Mark Converted
Mark Closed
Reschedule Follow Up
Take Over Manually
Add Note

Feedback section

RM should be able to submit feedback on AI scoring.

Fields:

Original classification
Corrected classification
Original score
Corrected score
Feedback type
Comment

Feedback types:

Score Wrong
Objection Wrong
Bad Response
Good Response
Other

Mock service

getRMLeadDetail(leadId)

Later real API:

GET /leads/:leadId

Mock response:

```
{
  "success": true,
  "data": {
    "lead": {
      "id": "lead_1",
      "name": "Amit Sharma",
      "phone": "9876543210",
      "email": "amit@example.com",
      "city": "Hyderabad",
      "role": "Financial Advisor",
      "networkSize": "25 to 30",
      "preferredLanguage": "Hinglish",
      "preferredChannel": "chat",
      "whatsappConsent": true,
      "status": "assigned_to_rm",
      "classification": "hot",
      "latestScore": 82,
      "latestSummary": "Lead asked for onboarding details and responded positively to daily payouts.",
      "latestNextAction": "RM should call today before 6 PM."
    },
    "campaign": {
      "id": "campaign_1",
      "name": "Demo Partner Campaign"
    },
    "latestConversation": {
```

```

    "id": "conv_1",
    "channel": "chat",
    "status": "completed",
    "language": "hinglish",
    "durationSeconds": 420,
    "summary": "Discussed brokerage share, daily payouts, and onboarding."
  },
  "messages": [
    {
      "id": "msg_1",
      "role": "user",
      "content": "I already have another broker.",
      "sequenceNo": 1,
      "timestamp": "2026-05-02T10:05:00Z"
    },
    {
      "id": "msg_2",
      "role": "assistant",
      "content": "That is actually a good sign because you already
understand this business. Are you getting 100 percent brokerage and daily
payouts?",
      "sequenceNo": 2,
      "timestamp": "2026-05-02T10:05:10Z"
    }
  ],
  "score": {
    "id": "score_1",
    "classification": "hot",
    "totalScore": 82,
    "readinessScore": 34,
    "engagementScore": 29,
    "fitScore": 19,
    "reason": "Asked for onboarding details, has relevant network, and
existing broker objection was resolved.",
    "recommendedNextAction": "RM should call today before 6 PM.",
    "positiveSignals": ["Asked for onboarding", "Has investor network",
"WhatsApp consent given"],
    "negativeSignals": [],
    "objections": [
      {
        "type": "Existing Broker",
        "status": "resolved",
        "leadStatement": "I already have another broker.",
        "aiResponseSummary": "Compared 100 percent brokerage and daily
payouts."
      }
    ]
  },
  "rmTask": {
    "id": "task_1",
    "status": "pending",
    "priority": "high",
    "recommendedAction": "Call today before 6 PM.",
    "suggestedOpeningLine": "Hi Amit, I saw you spoke with our AI assistant
about brokerage share and daily payouts."
  },
  "followUp": null

```



```
}  
}
```

12. Page 6: Follow Ups

Purpose

RM can see follow ups related to their assigned leads.

This is mostly for Warm leads or callback reminders.

Route

/rm/followups

Components

- Follow up list
- Status filters
- Message preview
- Open WhatsApp button
- Mark Sent button
- Reschedule optional

Columns

Lead name
Phone
Classification
Channel
Status
Message preview
Scheduled time
Actions

Actions

Open WhatsApp
Mark Sent
Reschedule
Cancel optional

Follow up status values

Ready
Opened
Sent
Cancelled

Mock service

```
getRMFollowUps (params)
openFollowUp (followUpId)
markFollowUpSent (followUpId)
```

Later APIs:

```
GET /followups
POST /followups/:followUpId/open
POST /followups/:followUpId/mark-sent
```

Mock response:

```
{
  "success": true,
  "data": [
    {
      "id": "follow_1",
      "lead": {
        "id": "lead_2",
        "name": "Priya Mehta",
        "phone": "9876543211",
        "classification": "warm"
      },
      "channel": "whatsapp",
      "status": "ready",
      "message": "Hi Priya, thanks for speaking with us. Here is the
onboarding link...",
      "waMeLink": "https://wa.me/919876543211?text=Hello",
      "scheduledAt": "2026-05-02T16:00:00Z",
      "createdAt": "2026-05-02T10:30:00Z"
    }
  ]
}
```

13. Page 7: Notifications

Purpose

RM can see system alerts.

Route

```
/rm/notifications
```

Notification types

New Hot Lead Assigned
Follow Up Due

Score Ready
Task Updated

Components

- Notification list
- Read/unread filter
- Mark all read button
- Click notification to open related lead

Notification card fields

Title
Message
Type
Read status
Created time
Related lead link

Mock services

```
getRMNotifications(params)
markNotificationRead(notificationId)
markAllNotificationsRead()
```

Later APIs:

```
GET /notifications
PATCH /notifications/:notificationId/read
PATCH /notifications/read-all
```

Mock response:

```
{
  "success": true,
  "data": [
    {
      "id": "notif_1",
      "title": "New Hot Lead Assigned",
      "message": "Amit Sharma has been classified as Hot and assigned to
you.",
      "type": "hot_lead_assigned",
      "isRead": false,
      "entityType": "lead",
      "entityId": "lead_1",
      "createdAt": "2026-05-02T10:25:00Z"
    }
  ]
}
```

14. Page 8: Feedback

Purpose

RM can view or submit feedback on AI scoring and conversation quality.

This is useful but not as important as Lead Detail and Hot Queue.

Route

`/rm/feedback`

Components

- Feedback history list
- Submit feedback form optional
- Lead selector optional
- Score correction form

Table columns

Lead name
Original classification
Corrected classification
Original score
Corrected score
Feedback type
Comment
Created date

Feedback form fields

Use React Hook Form plus Zod.

Fields:

Lead
Corrected classification
Corrected score optional
Feedback type
Comment

Validation:

Lead required
Feedback type required
Comment required

Mock services

```
getRMFeedback(params)
submitRMFeedback(payload)
```

Later APIs:

```
GET /feedback
POST /feedback
```

15. Optional Page: RM Performance

Purpose

Personal performance page for RM.

This is optional and should only be built if core pages are done.

Route

```
/rm/performance
```

Sections

```
Assigned leads
Converted leads
Closed leads
Pending leads
Conversion rate
Average score of assigned leads
Recent outcomes
```

16. Shared Components

Build reusable components.

Layout components

```
RMLayout
Sidebar
Header
PageHeader
Breadcrumbs optional
```

Data components

```
StatCard
StatusBadge
ClassificationBadge
LanguageBadge
ScoreBadge
```

EmptyState
LoadingSkeleton
ErrorState
DataTable
FilterBar
Pagination

Lead components

LeadCard
HotLeadCard
LeadProfileCard
ScoreCard
ScoreBreakdown
ObjectionCard
TranscriptViewer
RecommendedActionCard
SuggestedOpeningLineCard

Task components

RMTaskCard
TaskStatusBadge
TaskActionButtons

Follow up components

FollowUpCard
WhatsAppLinkButton
MessagePreview

Feedback components

FeedbackForm
FeedbackHistoryCard
ClassificationCorrectionSelect

17. Mock Data Requirements

Create static mock data for:

RM user
Client
Assigned Hot leads
Assigned Warm leads
Converted leads
Closed leads
RM tasks
Follow ups
Notifications
Conversations
Messages

Scores
Feedback

Minimum mock data:

One Hot lead with complete transcript and score
One Warm lead with WhatsApp follow up link
One converted lead
One closed lead
One unread notification
One feedback item

18. Zustand Store

Use Zustand only for local app state.

Store:

Auth user
Client
Sidebar collapsed state
Theme optional
Global selected filters optional

Do not store server data in Zustand. Server data should come through TanStack Query.

19. TanStack Query Hooks

Use TanStack Query for all mock service calls.

Examples:

```
useRMDashboard()  
useRMHotLeads(params)  
useRMLeads(params)  
useRMLeadDetail(leadId)  
useRMFollowUps(params)  
useRMNotifications(params)  
useRMFeedback(params)
```

Mock functions should:

Return promises
Simulate small delay optional
Return same response shape as real APIs

20. Loading, Empty, and Error States

Every main page should have:

Loading state

Use skeletons.

Empty states

Examples:

No Hot leads assigned
No follow ups due
No notifications
No feedback submitted
No leads found

Error state

Show:

Friendly message
Retry button

21. Visual Design Direction

RM Portal should feel:

Action focused
Clean
Fast to scan
Less complex than Admin Portal
Sales workflow oriented
Clear next action
Clear score and reason

Use shadcn/ui:

Card
Button
Badge
Table
Tabs
Dialog
Sheet
Dropdown Menu
Select
Input
Textarea
Skeleton
Separator
Tooltip
Alert
Progress

22. Most Important Demo Screens

If time is limited, prioritize:

- RM Dashboard
- Hot Leads
- Lead Detail
- Follow Ups
- Notifications

The Lead Detail page is the most important screen because it proves that the AI handoff has full context.

23. RM Portal Build Priority

Priority 1

- RM layout
- RM login
- Dashboard
- Hot lead queue
- Lead detail page
- Score card
- Transcript viewer
- Recommended action card
- Task actions

Priority 2

- Follow Ups page
- Notifications page
- My Leads page
- Mark converted / closed
- Mark follow up sent
- Feedback form

Priority 3

- Feedback history
- Performance page
- Extra filters
- Data chatbot optional
- Extra polish and animations

24. Final RM Portal Success Criteria

Shivansh's RM Portal is complete for MVP if:

- RM can login using mock auth
- Dashboard shows assigned Hot leads and follow ups
- Hot Leads page lists assigned Hot leads clearly
- Lead Detail page shows profile, score, reason, summary, objections, transcript, recommended next action, and suggested opening line
- RM can mark a lead converted or closed in mock UI
- Follow Ups page shows WhatsApp link and message preview
- Notifications page shows Hot lead assignment alert
- Feedback form exists for score correction
- UI uses static data but is ready for API integration
- All service functions return data in expected API response format